



Laboratorio 03: Clasificadores fundamentales

Regresión logística, SVM, SVM+RBF, Árbol de decisión y K-NN

Dr. Juan Bekios Calfa

Machine Learning

Universidad Católica del Norte

Propósito del laboratorio

Objetivo general

Construir, ajustar y comparar clasificadores fundamentales de Machine Learning para predecir niveles de deterioro cognitivo a partir de un conjunto de atributos binarios, utilizando modelos supervisados implementados con `scikit-learn`.

Objetivos específicos

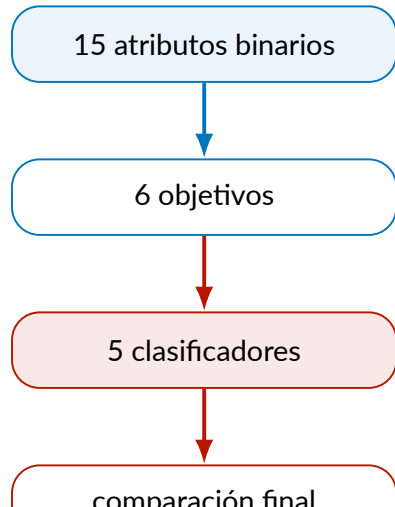
- Formular experimentos independientes usando las variables objetivo GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Implementar Regresión Logística, SVM lineal, SVM con kernel RBF, Árbol de Decisión y K-NN.
- Usar `Pipeline` y `GridSearchCV` para separar preprocesamiento, ajuste y evaluación.
- Comparar modelos mediante métricas globales, métricas macro y matrices de confusión.
- Elaborar un informe técnico reproducible siguiendo el formato indicado para el laboratorio.

Idea central del laboratorio

- El problema consiste en predecir agrupaciones de deterioro cognitivo a partir de 15 atributos binarios.
- El foco del laboratorio está en clasificadores fundamentales de Machine Learning.
- Cada columna objetivo define un experimento distinto.
- El estudiante debe comparar modelos, justificar hiperparámetros y discutir cuándo un modelo simple puede ser suficiente.

Mensaje clave

No gana necesariamente el modelo más complejo. Gana el modelo mejor validado, mejor interpretado y más coherente con los datos.

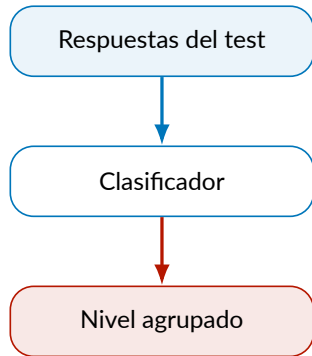


Problema de estudio

- Se trabajará con datos asociados a un test neuropsicológico reducido.
- Las entradas son respuestas binarias que representan aciertos o fallos en distintas preguntas.
- Las salidas corresponden a distintas formas de agrupar niveles de deterioro cognitivo.
- El objetivo del laboratorio es metodológico: construir, ajustar, evaluar e interpretar modelos supervisados.
- Los resultados no deben ser interpretados como diagnóstico clínico.

Pregunta computacional

¿Qué clasificador entrega el mejor equilibrio entre desempeño, interpretabilidad y estabilidad para cada forma de codificar el deterioro cognitivo?



Resumen del dataset

- El archivo contiene 1119 observaciones.
- Las variables de entrada corresponden a 15 atributos binarios.
- El identificador ID no debe utilizarse como variable predictora.
- Las columnas objetivo disponibles son GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.
- Cada columna objetivo se evaluará como un experimento independiente de clasificación supervisada.

0	1	0	1	0	1	0
1	0	1	0	1	0	1
0	1	0	1	0	1	0
1	0	1	0	1	0	1

Matriz de atributos binarios



GDS_Rk

Columna objetivo

Atributos de entrada

Atributos	Tipo
Día, Mes, Año, Estación	Binario
País, Ciudad, CalleLugar, NumeroPiso	Binario
Miguel2, González2, Avenida2, Imperial2	Binario
A682, Caldera2, Copiapo2	Binario

Implicancia metodológica

Al ser variables binarias, el preprocesamiento es simple. Sin embargo, modelos como SVM, Regresión Logística y K-NN deben implementarse dentro de un Pipeline con escalamiento para mantener una práctica experimental correcta.

Orientación temporal

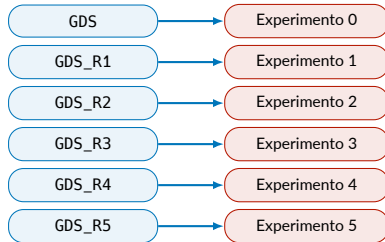
Orientación espacial

Memoria verbal

Memoria geográfica

Seis experimentos independientes

- Cada columna objetivo representa una forma distinta de agrupar la escala GDS.
- El conjunto de atributos de entrada se mantiene constante.
- Lo que cambia entre experimentos es la variable objetivo y el número de clases.
- Se debe reportar una tabla final por experimento y una comparación global.



Regla del laboratorio

No mezclar columnas objetivo en un mismo entrenamiento.
Cada objetivo se evalúa por separado.

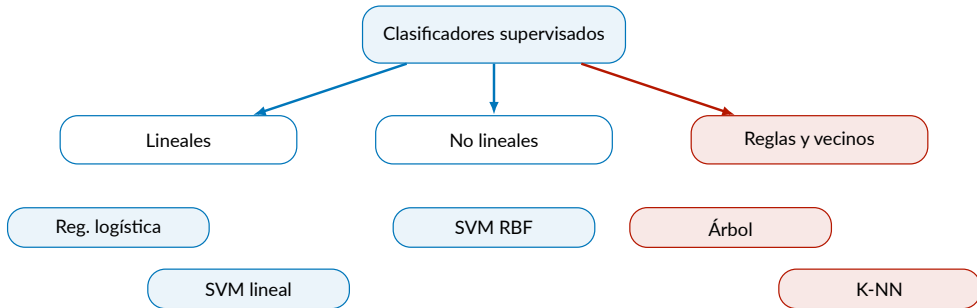
Modelos que se deben implementar



Comparación esperada

Los modelos representan familias distintas: clasificadores lineales, clasificadores con frontera no lineal, modelos basados en reglas y modelos basados en vecinos.

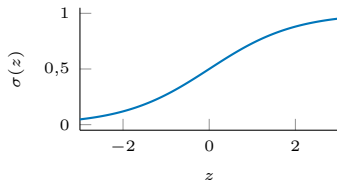
Mapa conceptual de los clasificadores



Regresión logística

- Modelo lineal para clasificación probabilística.
- Estima una probabilidad de pertenencia a cada clase.
- Permite interpretar coeficientes y dirección del efecto de cada atributo.
- Es una línea base fuerte cuando el problema es aproximadamente lineal.

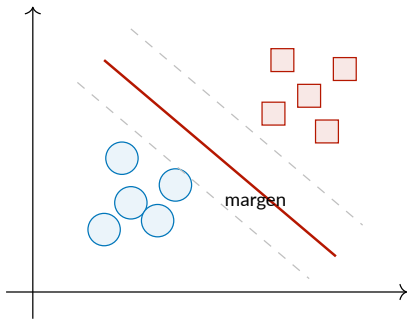
$$P(y = k \mid x) = \frac{e^{w_k^T x + b_k}}{\sum_j e^{w_j^T x + b_j}}$$



SVM lineal

- Busca una frontera de decisión que separe clases con el mayor margen posible.
- El hiperparámetro C controla el equilibrio entre margen amplio y errores de clasificación.
- Es útil cuando los datos pueden separarse razonablemente mediante fronteras lineales.

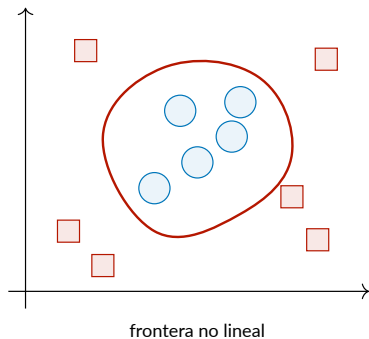
$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$



SVM con kernel RBF

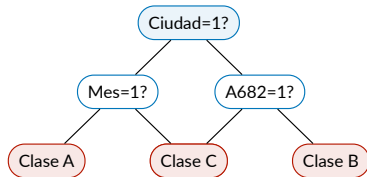
- Permite construir fronteras no lineales.
- El kernel RBF mide similitud local entre observaciones.
- C controla la penalización de errores.
- gamma controla la influencia local de cada punto.
- Puede funcionar bien cuando los patrones no son linealmente separables.

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$



Árbol de decisión

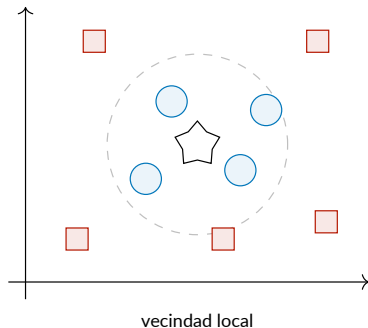
- Divide el espacio de atributos mediante reglas del tipo atributo $\leq$ umbral.
- Es muy interpretable, especialmente en problemas con variables binarias.
- Puede sobreajustar si no se limita la profundidad o el tamaño de las hojas.
- Se deben explorar hiperparámetros de regularización estructural.



K-NN

- Clasifica una nueva muestra observando las clases de sus vecinos más cercanos.
- Es simple, intuitivo y no paramétrico.
- Es sensible al escalamiento, a la elección de k y a la métrica de distancia.
- En datos binarios, se puede comparar distancia euclidiana y Manhattan.

$\hat{y}$ = voto mayoritario de los k vecinos

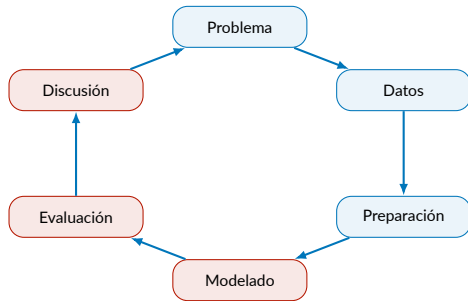


CRISP-DM como estructura del laboratorio

1. Comprensión del problema.
2. Comprensión de los datos.
3. Preparación de los datos.
4. Modelado con clasificadores fundamentales.
5. Evaluación comparativa.
6. Discusión y reproducibilidad.

Mensaje principal

La metodología permite justificar decisiones y no solo ejecutar modelos automáticamente.



Diseño experimental general

1. Seleccionar una columna objetivo: GDS, GDS_R1, ..., GDS_R5.
2. Separar X con los 15 atributos binarios e y con la columna objetivo activa.
3. Crear un conjunto de prueba externo estratificado, por ejemplo 20 %.
4. Ajustar hiperparámetros solamente sobre el conjunto de entrenamiento.
5. Usar validación cruzada interna con `StratifiedKFold`.
6. Evaluar el mejor modelo en el conjunto de prueba externo.
7. Repetir el proceso para los cinco clasificadores y las seis columnas objetivo.

Regla de honestidad experimental

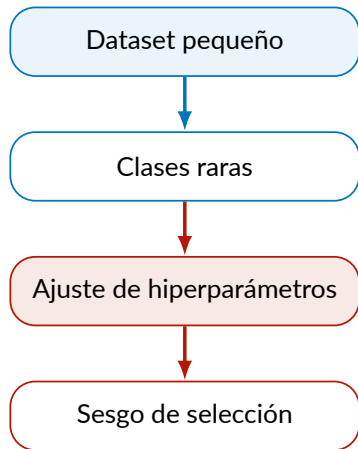
El conjunto de prueba externo no se usa para elegir hiperparámetros ni para decidir qué modelo conservar.

Problema de validación en este laboratorio

- El dataset tiene aproximadamente 1000 observaciones.
- Existe un fuerte desbalance entre clases.
- Algunas clases pueden tener solo 2 o 3 ejemplos.
- Una partición fija `train/test` puede dejar clases raras fuera del entrenamiento o fuera de la evaluación.
- Usar el mismo conjunto para elegir hiperparámetros y reportar resultados produce una estimación optimista.

Riesgo principal

El modelo puede parecer mejor de lo que realmente es porque los hiperparámetros fueron seleccionados usando la misma validación que luego se reporta como resultado final.



Por qué no basta con un k-fold simple

- La validación cruzada k-fold aprovecha mejor los datos que una partición fija.
- Sin embargo, si se usa el mismo k-fold para ajustar hiperparámetros y reportar el desempeño final, aparece sesgo.
- El problema aumenta cuando se prueban muchos modelos y muchas combinaciones de hiperparámetros.
- En este laboratorio se comparan 5 clasificadores sobre 6 variables objetivo, por lo que el riesgo de selección optimista es real.



Conclusión

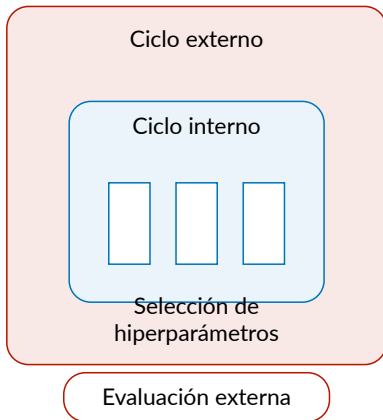
Un k-fold simple es mejor que un train/test fijo, pero no separa correctamente la selección de hiperparámetros de la evaluación final.

Validación cruzada anidada

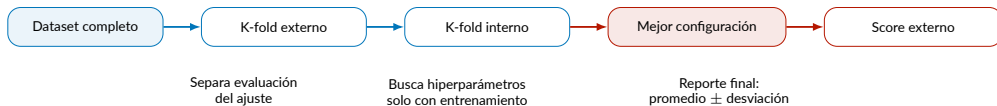
- La validación cruzada anidada separa dos tareas distintas.
- El ciclo externo estima el desempeño final del modelo.
- El ciclo interno selecciona hiperparámetros usando solo los datos de entrenamiento del fold externo.
- El fold externo de prueba nunca se usa para escoger hiperparámetros.
- El resultado final se reporta como promedio y desviación estándar sobre los folds externos.

Idea central

La selección del modelo ocurre dentro del entrenamiento. La evaluación final ocurre fuera de esa selección.



Flujo de datos y evaluación



Validación recomendada

Usar validación cruzada anidada. El ciclo externo estima el desempeño final, mientras que el ciclo interno se usa exclusivamente para seleccionar hiperparámetros mediante `GridSearchCV`.

Advertencia

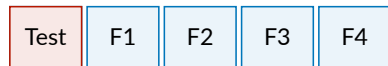
El resultado de `GridSearchCV` no debe reportarse como desempeño final. Ese resultado solo sirve para elegir la configuración dentro de cada fold externo.

Ciclo externo: estimación del desempeño

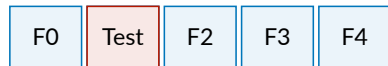
1. Dividir el dataset en k folds estratificados.
2. En cada iteración, dejar un fold como prueba externa.
3. Usar los folds restantes como entrenamiento externo.
4. Dentro del entrenamiento externo, buscar hiperparámetros.
5. Evaluar la configuración seleccionada en el fold externo que quedó fuera.
6. Repetir hasta que todos los folds hayan sido usados como prueba externa.

Resultado

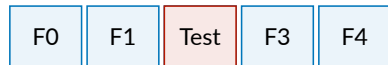
Se obtiene un conjunto de scores externos, uno por fold. El desempeño final se reporta como promedio y desviación estándar.



Iteración 1



Iteración 2



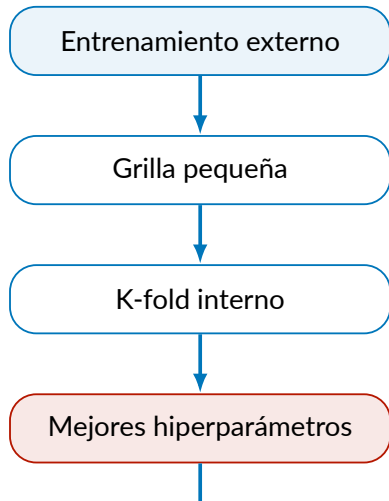
Iteración 3

Ciclo interno: selección de hiperparámetros

- El ciclo interno solo usa los datos de entrenamiento del fold externo.
- Allí se prueban distintas combinaciones de hiperparámetros.
- Se selecciona la combinación con mejor desempeño promedio interno.
- Luego se reentrena el modelo con todo el entrenamiento externo.
- Finalmente se evalúa una sola vez en el fold externo de prueba.

Regla clave

El fold externo de prueba no participa en la selección de hiperparámetros.



Cómo elegir el número de folds

- El número de folds no debe elegirse solo por costumbre.
- En datasets desbalanceados, el valor de k está limitado por la clase con menos ejemplos.
- Si una clase tiene solo 2 ejemplos, no se puede usar $k=5$ de forma estratificada.
- Si una clase tiene solo 3 ejemplos, no se recomienda usar más de 3 folds externos.

$$k_{\text{externo}} \leq n_{\text{mín}}$$

Criterio práctico

Usar:

$$k_{\text{externo}} = \min(5, n_{\text{mín}})$$

donde $n_{\text{mín}}$ es el número de ejemplos de la clase menos representada.

Caso crítico

Si alguna clase tiene solo 1 ejemplo, no es posible garantizar validación estratificada para esa clase. El informe debe declarar esta limitación y justificar si se agrupa, se excluye del análisis o se reporta como caso exploratorio.

Problemas específicos para este caso

- **Folds inestables:** con clases de 2 o 3 casos, pequeños cambios en los folds pueden cambiar mucho el resultado.
- **Métricas macro variables:** F1 macro puede fluctuar fuertemente cuando una clase rara se predice mal.
- **Matriz de confusión difícil de interpretar:** una clase con 2 ejemplos no permite estimar errores de forma robusta.
- **SVM RBF y K-NN sensibles:** pueden ajustarse demasiado a pocos ejemplos minoritarios.
- **Árboles de decisión:** pueden crear reglas específicas para casos raros y sobreajustar.
- **Regresión logística y SVM lineal:** pueden ser más estables, pero también pueden ignorar clases muy minoritarias.

Advertencia metodológica

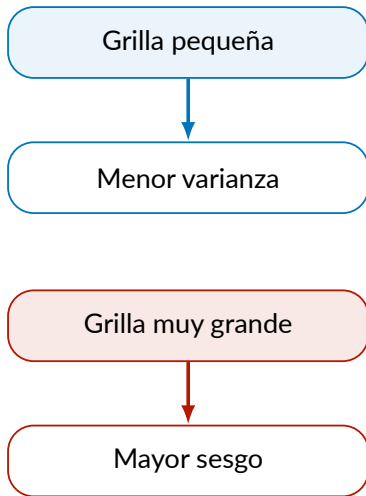
Si las clases raras tienen muy pocos ejemplos, no se debe concluir que un modelo es clínicamente confiable para esas clases. Solo se puede hablar de evidencia experimental limitada.

Grillas pequeñas para reducir sesgo de selección

- Mientras más hiperparámetros se prueban, mayor es el riesgo de seleccionar una configuración que ganó por azar.
- En datasets pequeños y desbalanceados, conviene usar grillas reducidas.
- La grilla debe estar justificada por conocimiento del modelo, no por búsqueda exhaustiva excesiva.
- La comparación debe favorecer modelos estables, no solo modelos con el mayor score puntual.

Recomendación

Usar una grilla conservadora y reportar no solo el promedio, sino también la desviación estándar entre folds externos.



Protocolo recomendado para el laboratorio

1. Para cada variable objetivo, calcular la distribución de clases.
2. Determinar $n_{\text{mín}}$, es decir, la cantidad de ejemplos de la clase menos frecuente.
3. Definir $k_{\text{externo}} = \text{mín}(5, n_{\text{mín}})$.
4. Usar validación cruzada anidada cuando el número de ejemplos por clase lo permita.
5. En el ciclo interno, usar una grilla pequeña de hiperparámetros.
6. Reportar el desempeño usando los folds externos.
7. Presentar promedio, desviación estándar, matriz de confusión agregada y métricas por clase.
8. Declarar explícitamente las clases con soporte muy bajo.

Resultado esperado

El informe debe mostrar no solo cuál modelo obtiene mejor resultado, sino también cuán estable y confiable es ese resultado bajo desbalance severo.

Implementación sugerida en scikit-learn

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV, cross_validate
from sklearn.metrics import make_scorer, f1_score, balanced_accuracy_score
import numpy as np

# y corresponde a la columna objetivo activa
class_counts = y.value_counts()
n_min = class_counts.min()

k_outer = min(5, n_min)

if k_outer < 2:
    raise ValueError("No es posible aplicar k-fold estratificado: existe una clase con un solo ejemplo.")

outer_cv = StratifiedKFold(
    n_splits=k_outer,
    shuffle=True,
    random_state=42
)

# El ciclo interno debe ser menor o igual al externo y compatible con las clases
k_inner = max(2, min(3, k_outer))

inner_cv = StratifiedKFold(
    n_splits=k_inner,
    shuffle=True,
    random_state=123
)
```

Qué reportar en los resultados

- Distribución de clases para cada variable objetivo.
- Valor de $n_{\min}$ y número de folds usado en cada experimento.
- Promedio y desviación estándar de F1 macro en los folds externos.
- Balanced accuracy como métrica complementaria.
- Precision, recall y F1 por clase, incluyendo el soporte de cada clase.
- Matriz de confusión agregada sobre predicciones externas.
- Hiperparámetros seleccionados con mayor frecuencia.
- Comentario explícito sobre clases con 1, 2 o 3 ejemplos.

Advertencia para la interpretación

Una clase con soporte muy bajo no permite conclusiones robustas. Los resultados de esa clase deben interpretarse como evidencia exploratoria.

Decisiones metodológicas aceptables

- **Mantener todas las clases:** permite respetar la formulación original, pero puede generar resultados inestables.
- **Agrupar clases raras:** puede mejorar estabilidad, pero debe justificarse desde el significado del problema.
- **Reportar el experimento como exploratorio:** adecuado cuando una clase tiene muy pocos ejemplos para validar.
- **Usar grillas reducidas:** disminuye el riesgo de sesgo por selección de hiperparámetros.
- **Comparar modelos simples y complejos:** permite observar si la complejidad realmente aporta.

Criterio de calidad

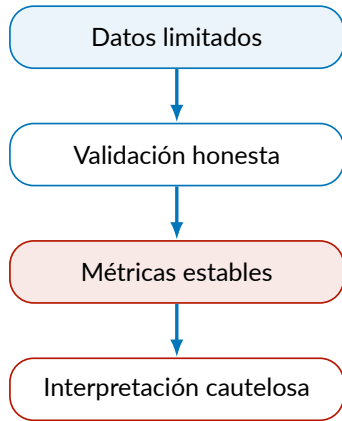
La mejor decisión no es necesariamente la que entrega el mayor score, sino la que produce una comparación más honesta, reproducible e interpretable.

Mensaje final sobre validación

- La validación debe reflejar las limitaciones reales del dataset.
- El desbalance extremo impide conclusiones fuertes para clases con muy pocos ejemplos.
- La validación cruzada anidada reduce el sesgo por selección de hiperparámetros.
- Aun así, los resultados deben interpretarse con cautela cuando el soporte por clase es bajo.

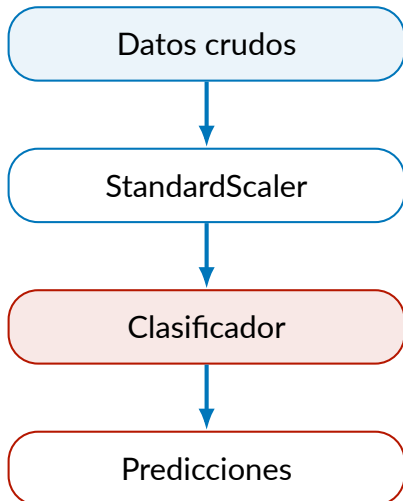
Conclusión

El laboratorio debe evaluar modelos de forma honesta, no solo buscar el mayor valor de una métrica.



Uso obligatorio de Pipeline

- Un Pipeline evita fuga de información entre entrenamiento y validación.
- El escalamiento se ajusta solo con los datos de entrenamiento de cada fold.
- La misma estructura permite comparar modelos de forma ordenada.
- Para Árboles, el escalamiento no es necesario, pero puede dejarse como passthrough o usarse una tubería separada.



Preparación de datos

```
feature_cols = [  
    "Dia", "Mes", "Anio", "Estacion", "Pais", "Ciudad", "CalleLugar",  
    "NumeroPiso", "Miguel2", "Gonzalez2", "Avenida2", "Imperial2",  
    "A682", "Caldera2", "Copiapo2"  
]  
  
targets = ["GDS", "GDS_R1", "GDS_R2", "GDS_R3", "GDS_R4", "GDS_R5"]  
  
target_name = "GDS_R2" # Cambiar en cada experimento  
  
X = df[feature_cols].astype(float)  
y = df[target_name].astype(int)
```

Resultado esperado

El mismo bloque debe poder ejecutarse para cualquiera de las seis columnas objetivo, sin duplicar código innecesario.

Pipelines base de clasificadores

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

pipelines = {
    "logistic": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, class_weight="balanced"))
    ]),
    "svm_linear": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", SVC(kernel="linear", class_weight="balanced"))
    ]),
    "svm_rbf": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", SVC(kernel="rbf", class_weight="balanced"))
    ]),
    "tree": Pipeline([
        ("clf", DecisionTreeClassifier(class_weight="balanced", random_state=42))
    ]),
    "knn": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", KNeighborsClassifier())
    ])
}
```

Hiperparámetros a explorar

Modelo	Parámetro	Valores sugeridos
Regresión Logística	C	0,01, 0,1, 1, 10
SVM lineal	C	0,01, 0,1, 1, 10
SVM RBF	C, gamma	C: 0,1, 1, 10; gamma: scale, 0,01, 0,1, 1
Árbol de decisión	max_depth, min_samples_leaf	max_depth: 2, 3, 4, 5, None; hojas: 1, 3, 5, 10
K-NN	n_neighbors, weights, metric	vecinos: 3, 5, 7, 9, 11; pesos: uniform, distance; métrica: euclidean, manhattan

Cuidado

Una grilla grande no siempre es mejor. Debe ser razonable, ejecutable y justificada en el informe.

Ejemplo de grilla para GridSearchCV

```
param_grids = {  
    "logistic": {"clf__C": [0.01, 0.1, 1, 10]},  
    "svm_linear": {"clf__C": [0.01, 0.1, 1, 10]},  
    "svm_rbf": {  
        "clf__C": [0.1, 1, 10],  
        "clf__gamma": ["scale", 0.01, 0.1, 1]  
    },  
    "tree": {  
        "clf__max_depth": [2, 3, 4, 5, None],  
        "clf__min_samples_leaf": [1, 3, 5, 10]  
    },  
    "knn": {  
        "clf__n_neighbors": [3, 5, 7, 9, 11],  
        "clf__weights": ["uniform", "distance"],  
        "clf__metric": ["euclidean", "manhattan"]  
    }  
}
```

Ejecución del experimento

```
from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search = GridSearchCV(
    estimator=pipelines[model_name],
    param_grid=param_grids[model_name],
    scoring="f1_macro",
    cv=cv,
    n_jobs=-1,
    refit=True
)

search.fit(X_train, y_train)
y_pred = search.predict(X_test)
```

Métrica para seleccionar modelo

Se recomienda seleccionar hiperparámetros con `f1_macro`, porque da importancia a clases minoritarias.

Métricas obligatorias

- **Accuracy:** proporción total de aciertos.
- **Precision macro:** promedio de precisión por clase.
- **Recall macro:** promedio de sensibilidad por clase.
- **F1 macro:** equilibrio entre precision y recall por clase.
- **Matriz de confusión:** distribución de errores entre clases.

Predicciones		
Real	TP	FN
	FP	TN

Reporte mínimo

Reportar métricas por modelo, por objetivo y por clase cuando corresponda.

Por qué usar F1 macro

- En problemas con clases desbalanceadas, la accuracy puede ser engañosa.
- F1 macro calcula el F1 de cada clase y luego promedia.
- De esta forma, las clases minoritarias no desaparecen dentro del promedio global.

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^K F1_k$$

Interpretación

Un modelo con alta accuracy pero bajo F1 macro puede estar prediciendo bien solo la clase mayoritaria.

Matriz de confusión

- Permite observar en qué clases se equivoca el modelo.
- Es clave para analizar confusiones entre niveles cercanos de deterioro.
- Debe incluirse al menos para el mejor modelo de cada experimento.
- Puede normalizarse por fila para facilitar la interpretación.

		Predicho		
Real	Clase 0	Clase 1	Clase 2	
	70	15	15	
	15	70	15	
	15	15	70	
		Clase 0	Clase 1	Clase 2

Cómo condensar las métricas para comparar modelos

- En este laboratorio no basta con comparar modelos usando una sola métrica.
- Se recomienda reportar varias métricas: F1 macro, balanced accuracy, precision macro, recall macro y estabilidad entre folds.
- Para comparar modelos de forma resumida, se puede construir un **Índice Comparativo Normalizado**.
- Este índice no reemplaza el análisis detallado, pero ayuda a ordenar modelos en una tabla final.

$$ICN = 0,40 \cdot \widetilde{F1}_{macro} + 0,25 \cdot \widetilde{BA} + 0,20 \cdot \widetilde{Recall}_{macro} + 0,10 \cdot \widetilde{Precision}_{macro} + 0,05 \cdot \widetilde{Estabilidad}$$

Interpretación

Cada métrica se normaliza entre 0 y 1. Un valor mayor de **ICN** indica un mejor equilibrio global entre desempeño, sensibilidad a clases minoritarias y estabilidad.

Advertencia

El índice compuesto debe usarse solo como resumen comparativo. La decisión final debe considerar también la matriz de confusión, el soporte por clase y el comportamiento en clases con pocos ejemplos.

Normalización de métricas

- Para combinar métricas distintas, primero deben llevarse a una escala común.
- Para métricas donde un valor mayor es mejor, se usa normalización min-max.
- Para métricas donde un valor menor es mejor, como la desviación estándar entre folds, se invierte la escala.
- Esto permite comparar modelos mediante un puntaje resumen.

Mayor es mejor

$$\tilde{m}_i = \frac{m_i - \min(m)}{\max(m) - \min(m) + \epsilon}$$

Menor es mejor

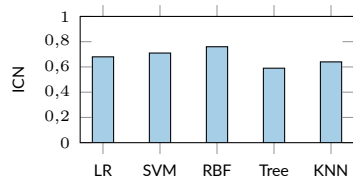
$$\tilde{m}_i = 1 - \frac{m_i - \min(m)}{\max(m) - \min(m) + \epsilon}$$

Ejemplo

Si se usa la desviación estándar del F1 macro entre folds, un modelo con menor desviación recibe mayor puntaje de estabilidad.

Visualizaciones obligatorias

- Distribución de clases para cada columna objetivo.
- Comparación de $F1$ macro entre modelos.
- Comparación de balanced accuracy entre modelos.
- Matriz de confusión del mejor modelo por objetivo.
- Tabla o mapa de calor con todas las métricas normalizadas.
- Gráfico resumen del **Índice Comparativo Normalizado**.



Lectura del gráfico

El **ICN** resume el comportamiento global de cada modelo. Sin embargo, el informe debe explicar qué métricas favorecieron o perjudicaron a cada clasificador.

Tabla resumen para comparar modelos

Modelo	F1 macro	Balanced Acc.	Recall macro	Precision macro	Estabilidad	ICN
Regresión Logística	0.62	0.65	0.61	0.64	0.82	0.68
SVM lineal	0.64	0.66	0.63	0.65	0.84	0.71
SVM RBF	0.69	0.70	0.67	0.68	0.78	0.76
Árbol de decisión	0.58	0.57	0.55	0.60	0.62	0.59
K-NN	0.61	0.62	0.60	0.61	0.70	0.64

Uso recomendado

Construir una tabla como esta para cada variable objetivo: GDS, GDS_R1, GDS_R2, GDS_R3, GDS_R4 y GDS_R5.

Cuidado

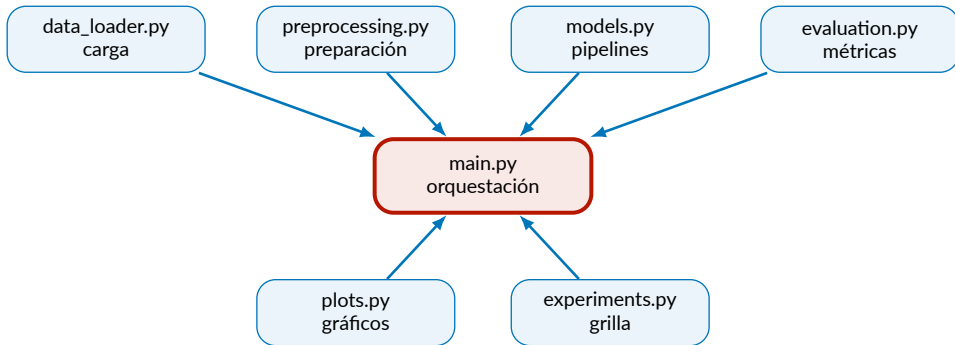
Los valores de esta tabla son solo ilustrativos. En el informe deben reemplazarse por los resultados reales obtenidos mediante validación cruzada anidada.

Mapa de calor de métricas normalizadas

- El mapa de calor permite comparar rápidamente modelos y métricas.
- Cada celda representa una métrica normalizada entre 0 y 1.
- Las filas corresponden a modelos.
- Las columnas corresponden a métricas.
- El informe debe acompañar el mapa con una interpretación escrita.

	F1	BA	Recall	Precision	Estab.	ICN
LR	0.62	0.65	0.61	0.64	0.82	0.68
SVM	0.64	0.66	0.63	0.65	0.84	0.71
RBF	0.69	0.70	0.67	0.68	0.78	0.76
Tree	0.58	0.57	0.55	0.60	0.62	0.59
KNN	0.61	0.62	0.60	0.61	0.70	0.64

Estructura modular recomendada



La estructura debe permitir repetir experimentos sin copiar y pegar bloques grandes de código.

Jerarquía sugerida de directorios

Estructura

```
laboratorio_03_clasificadores/  
|-- data/  
    |-- raw/  
    |-- processed/  
|-- outputs/  
    |-- figures/  
    |-- tables/  
|-- src/  
    |-- main.py  
    |-- data_loader.py  
    |-- preprocessing.py  
    |-- models.py  
    |-- experiments.py  
    |-- evaluation.py  
    |-- plots.py  
|-- environment.yml  
|-- README.md  
-- .gitignore
```

Lectura

- src/: implementación.
- outputs/: resultados.
- README.md: ejecución.
- environment.yml: entorno.

Entorno de trabajo sugerido

```
conda create -n lab03_ml python=3.11
conda activate lab03_ml
pip install numpy pandas scikit-learn matplotlib seaborn pyreadstat jupyter
```

- Se recomienda trabajar en Visual Studio Code o Google Colab.
- Si se usa VS Code, seleccionar el intérprete del entorno `lab03_ml`.
- El proyecto debe poder ejecutarse desde un archivo principal o notebook claramente documentado.

Experimentos mínimos requeridos

1. Ejecutar los cinco modelos sobre GDS.
2. Ejecutar los cinco modelos sobre GDS_R1.
3. Ejecutar los cinco modelos sobre GDS_R2.
4. Ejecutar los cinco modelos sobre GDS_R3.
5. Ejecutar los cinco modelos sobre GDS_R4.
6. Ejecutar los cinco modelos sobre GDS_R5.
7. Comparar los 30 resultados principales usando F1 macro.
8. Seleccionar el mejor modelo por objetivo y justificar por qué.

Resultado final esperado

Una comparación completa de clasificadores fundamentales aplicada a distintas formulaciones del mismo problema.

Preguntas que debe responder el informe

- ¿Cuál clasificador obtiene el mejor F1 macro para cada columna objetivo?
- ¿La Regresión Logística es competitiva frente a modelos no lineales?
- ¿El kernel RBF mejora realmente al SVM lineal?
- ¿El Árbol de decisión entrega reglas interpretables sin perder demasiado desempeño?
- ¿K-NN es estable o depende demasiado de k y de la distancia?
- ¿Qué objetivo parece más difícil de predecir y por qué?
- ¿Existen clases con recall muy bajo?
- ¿Hay evidencia de sobreajuste en algún modelo?
- ¿Qué decisión metodológica influyó más en los resultados?

Formato del informe de laboratorio

- El informe debe entregarse en PDF siguiendo el formato oficial indicado para el laboratorio.
- Debe incluir portada con título, asignatura, profesor, integrantes, carrera y correos.
- Debe incluir resumen breve, introducción, marco teórico, metodología, resultados, discusión, conclusiones, código y reproducibilidad.
- Las tablas y figuras deben estar numeradas, tituladas y explicadas en el texto.
- El informe no debe ser una copia del código: debe interpretar los resultados.

Extensión sugerida

Mantener las secciones principales dentro de los límites indicados por el formato del informe. Usar anexos solo para tablas largas o detalles complementarios.

Contenido esperado por sección

Sección	Contenido esperado
Introducción	Contexto del problema, objetivo del laboratorio y relevancia de comparar clasificadores.
Marco teórico	Descripción breve de Regresión Logística, SVM, kernel RBF, Árboles y K-NN.
Metodología	Dataset, variables, preprocesamiento, partición, validación, grillas e implementación.
Resultados	Tablas de métricas, gráficos comparativos y matrices de confusión.
Discusión	Interpretación crítica, comparación de modelos, errores y limitaciones.
Conclusiones	Hallazgos principales, recomendaciones y trabajo futuro.
Código	Estructura del proyecto, instrucciones de ejecución y reproducibilidad.

Entregables

- Informe de laboratorio en formato PDF.
- Código fuente modular en archivo `.zip`.
- Repositorio GitHub con acceso para profesor y ayudantes.
- Carpeta `outputs/` con tablas, figuras y resultados generados.
- Archivo `README.md` con instrucciones para ejecutar el laboratorio.
- Archivo de entorno: `environment.yml` o `requirements.txt`.

Condición de reproducibilidad

El corrector debe poder ejecutar el proyecto y obtener las mismas tablas principales reportadas en el informe.

Criterios de evaluación propuestos

Criterio	%
Comprensión del problema y formulación experimental	15
Implementación correcta de clasificadores y pipelines	25
Diseño de validación, ajuste de hiperparámetros y reproducibilidad	20
Análisis de métricas, matrices de confusión y comparación de modelos	20
Calidad del informe, discusión, conclusiones y presentación visual	20

Observación

La nota no depende solo del mejor número obtenido, sino de la calidad del diseño experimental y de la interpretación.

Errores comunes que se deben evitar

- Usar el identificador ID como variable predictora.
- Ajustar hiperparámetros mirando el conjunto de prueba.
- Reportar solo accuracy en un problema con clases desbalanceadas.
- Comparar modelos sin usar la misma partición o la misma métrica.
- Copiar código sin explicar decisiones metodológicas.
- No incluir matrices de confusión ni análisis por clase.
- Entregar resultados no reproducibles o sin instrucciones de ejecución.

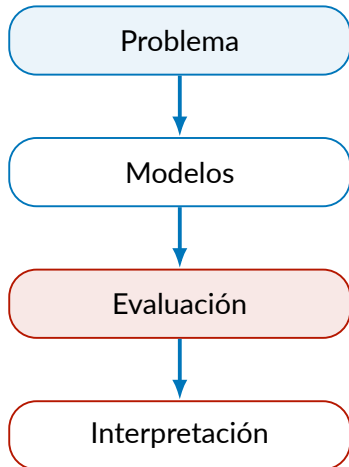
Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Implementar clasificadores fundamentales usando `scikit-learn`.
- Construir `Pipeline` y `GridSearchCV` sin fuga de información.
- Comparar modelos lineales, no lineales, basados en reglas y basados en vecinos.
- Interpretar métricas macro y matrices de confusión.
- Elaborar un informe técnico claro, visual y reproducible.

Cierre del laboratorio

- El mismo dataset puede generar conclusiones distintas según la formulación del objetivo.
- Los clasificadores fundamentales permiten entender sesgo, varianza, interpretabilidad y complejidad.
- La comparación honesta requiere validación, métricas adecuadas y análisis por clase.
- El informe debe demostrar comprensión, no solo ejecución de código.



¿Preguntas?